# UPPAAL Verification Screenshots — 1M1S AMBA AHB

23ECE337 System on Chip · companion to *1M1S_Report.pdf* · UPPAAL 5.0.0 (academic licence)

C. Deepthi Sree (BL.EN.U4ECE23107), G. Sohan (BL.EN.U4ECE23117), K. Lochana Reddy (BL.EN.U4ECE23127)

| Part | Model opened in UPPAAL | Query file imported | Figures |
|---|---|---|---|
| A. Model | `model/demo_view/amba_AHB_1M1S_clean.uppaal.xml` | — | 1–4 |
| B. Paper properties | `model/demo_view/amba_AHB_1M1S_clean.uppaal.xml` | `queries/demo_view/1M1S_QUERIES.q` | 5–6 |
| C. Communication in time | `model/demo_view/amba_AHB_1M1S_timed_clean.uppaal.xml` | `queries/demo_view/1M1S_TIMED_QUERIES.q` | 7–9 |
| D. Bug study | `model/demo_view/amba_AHB_1M1S_timed_noreset_clean.uppaal.xml` | `queries/demo_view/BUG_STUDY_NORESET.q` | 10–16 |

The demo copies differ from the models used for the report only in two cosmetic ways: the source-line comment boxes are removed, and "$" in names is written "__" because the UPPAAL 5.0 GUI cannot handle "$". They give the same verification results (checked with verifyta).

## A. The 1M1S model in the UPPAAL editor

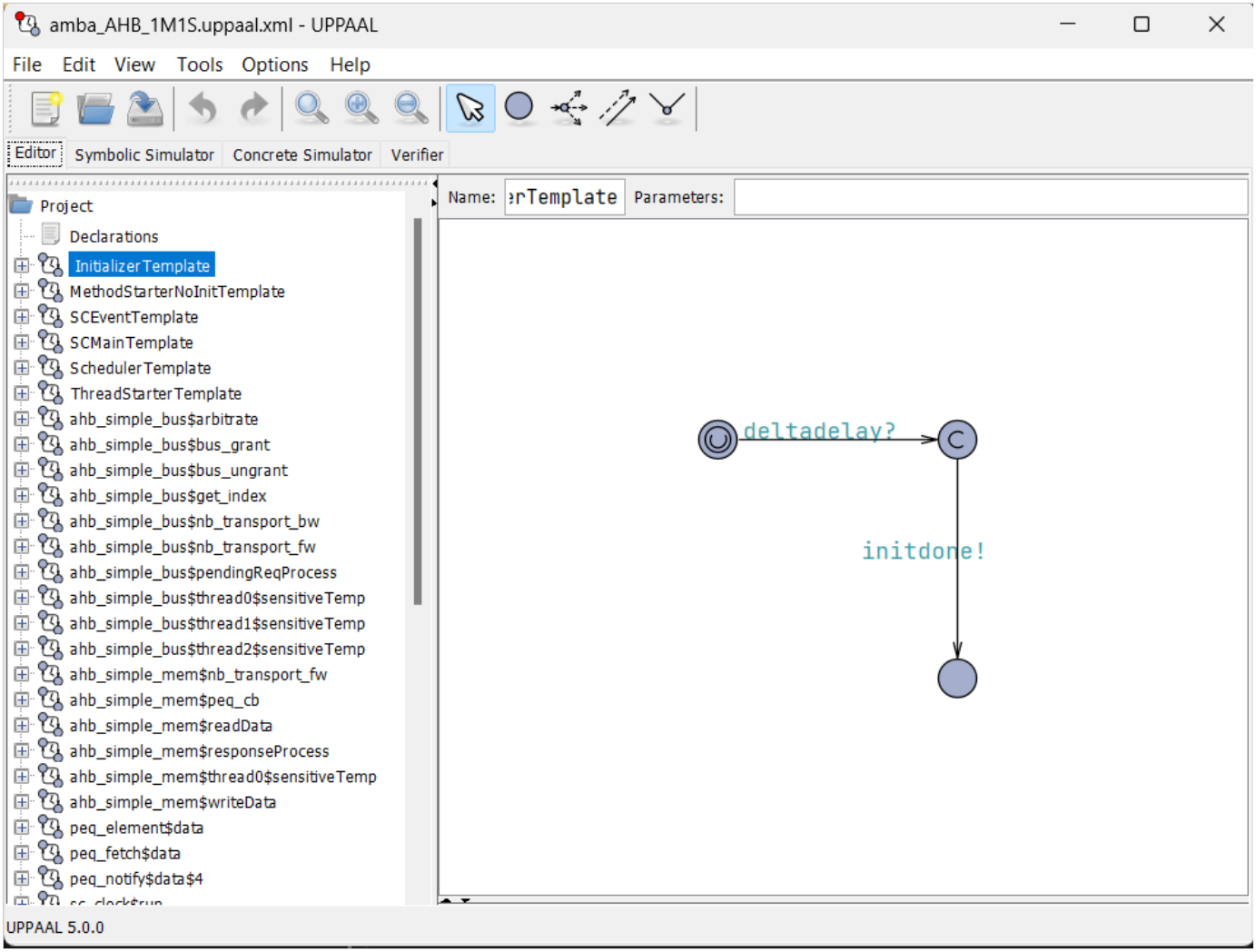



**Fig. 1.** The 1M1S model opened in UPPAAL 5.0.0. The left panel lists the 44 templates: SystemC scheduler and processes, bus (`ahb_simple_bus`), memory slave (`ahb_simple_mem`), PEQ automata and master adapter.

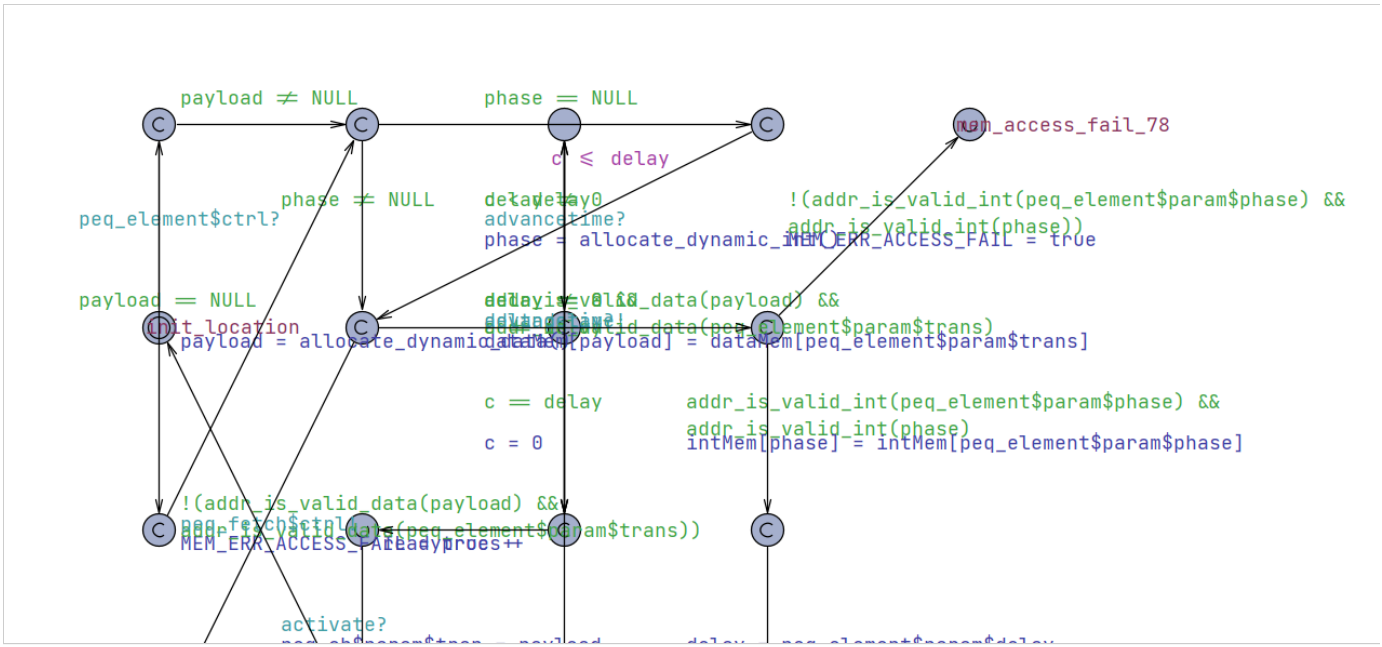



**Fig. 2.** `peq_element`: the PEQ notification automaton with local clock `c` and invariant `c ≤ delay` (paper Fig. 4). It stores the payload and phase, waits until `c == delay`, then signals the scheduler. The `mem_access_fail` branches are pointer-safety checks.

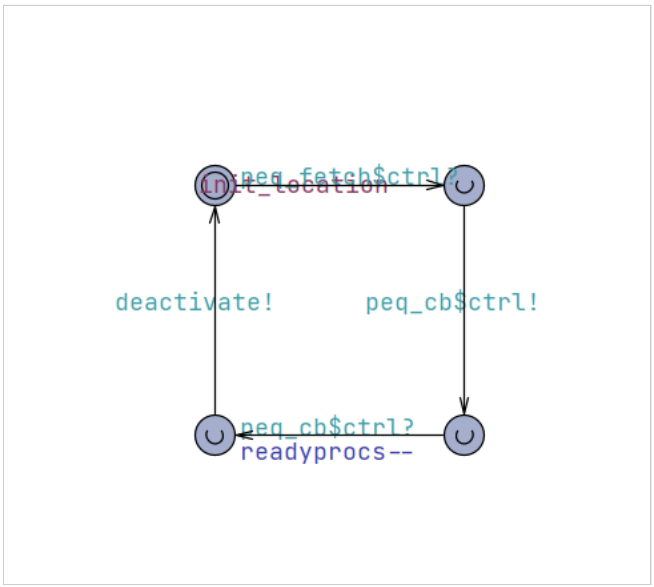



**Fig. 3.** `peq_fetch`: the callback invocation automaton (paper Fig. 5).

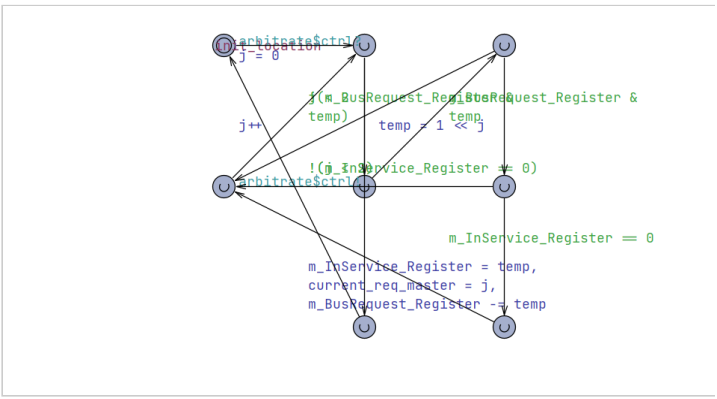



**Fig. 4.** `ahb_simple_bus__arbitrate`: grants the bus to a requesting master only when `m_InService_Register == 0`.

## B. Paper properties 1 and 3 on the 1M1S model

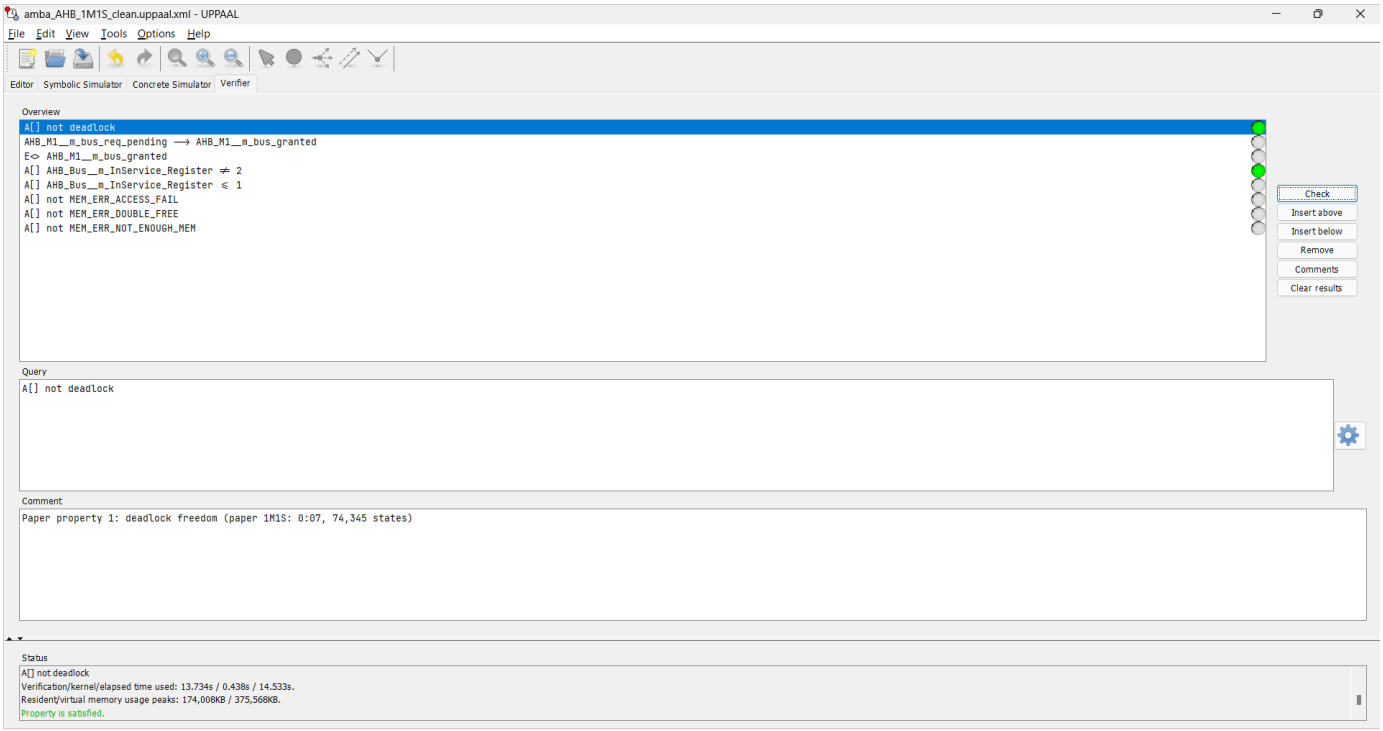



**Fig. 5.** `A[] not deadlock`: **Property is satisfied** (13.7 s in the GUI). The command-line run explored 1,241,387 states; the paper reports 74,345.

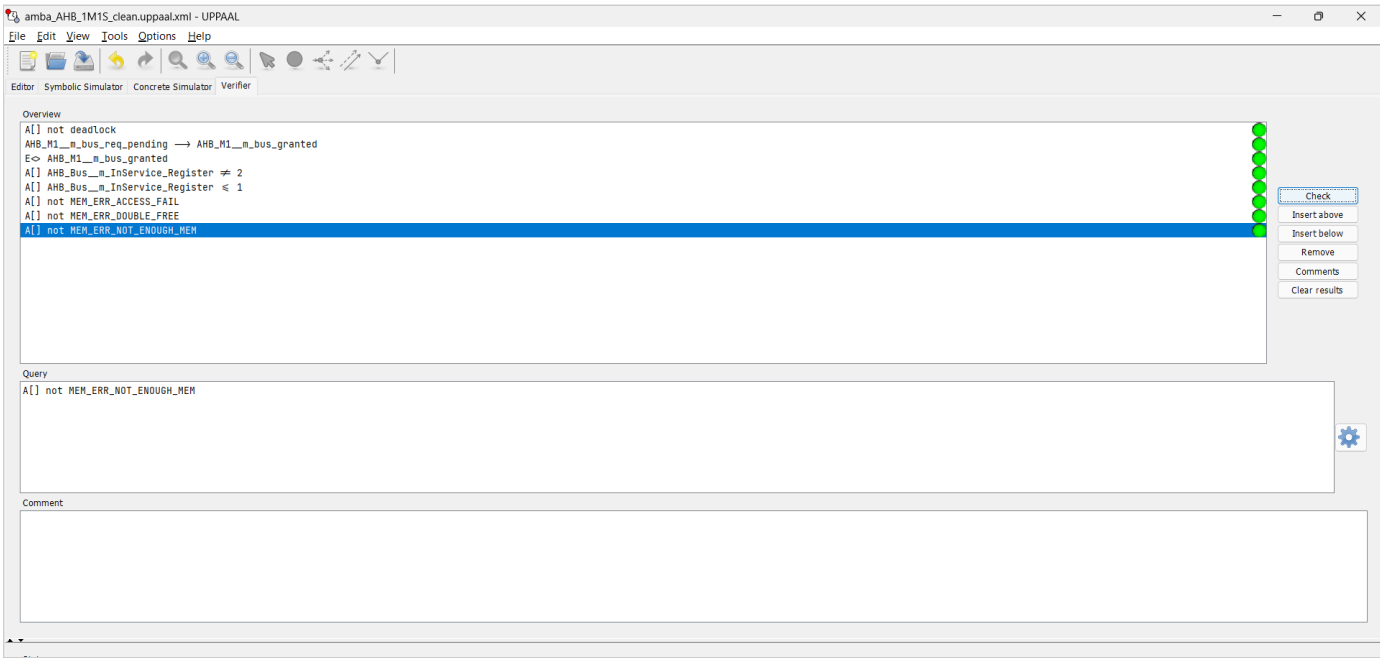



**Fig. 6.** All eight 1M1S queries **satisfied**: deadlock freedom; bus grant to M1 as a leads-to property (`AHB_M1__m_bus_req_pending --> AHB_M1__m_bus_granted`); reachability of a grant; the arbiter never grants the unused master-2 slot and its register stays ≤ 1; and no memory errors.

## C. Paper property 4: communication in time

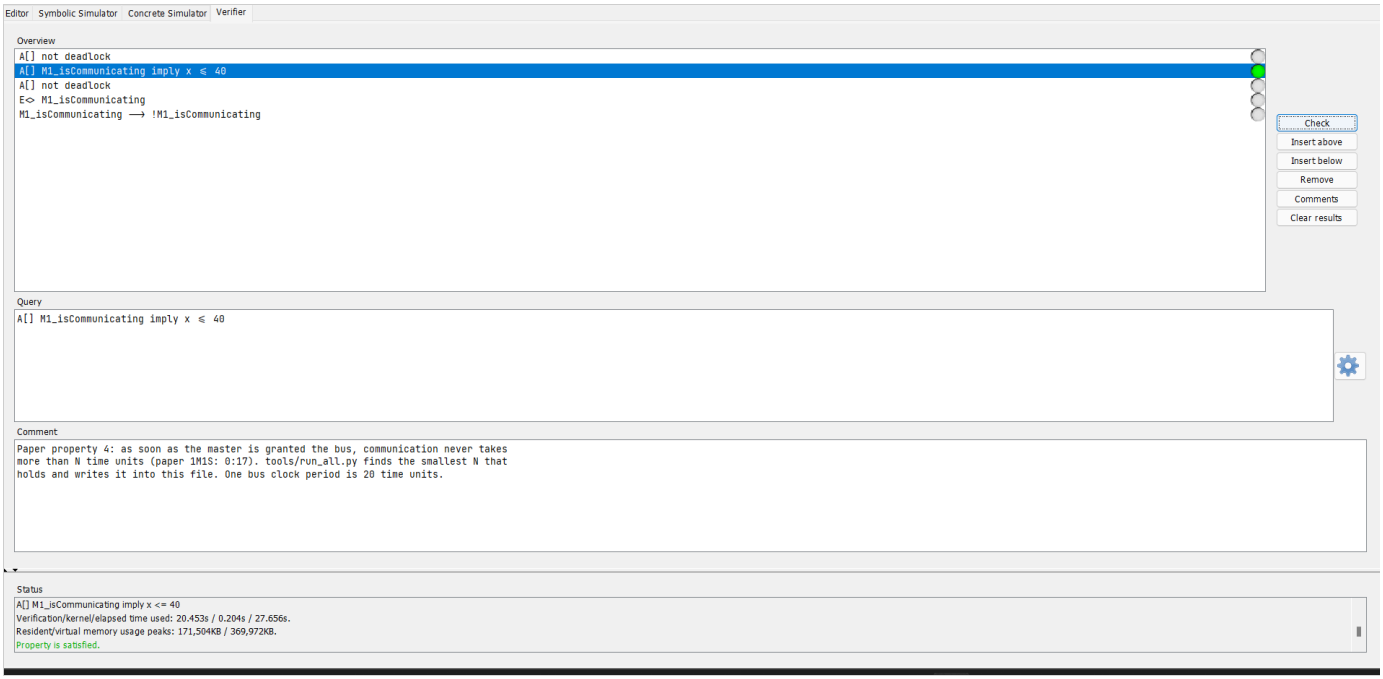



**Fig. 7.** `A[] M1_isCommunicating imply x ≤ 40`: **Property is satisfied** (20.5 s). Clock `x` is reset at every bus grant, as in the paper.

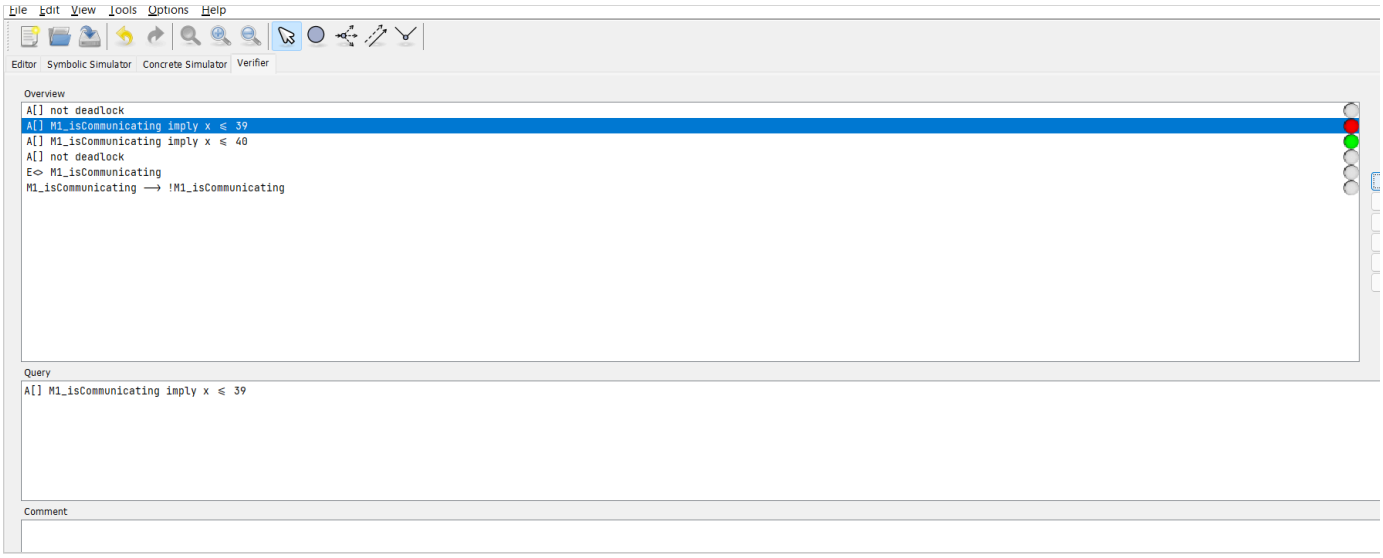



**Fig. 8.** `x ≤ 39` is **not satisfied** while `x ≤ 40` is satisfied, so N = 40 time units (2 clock periods of 20) is the smallest bound.

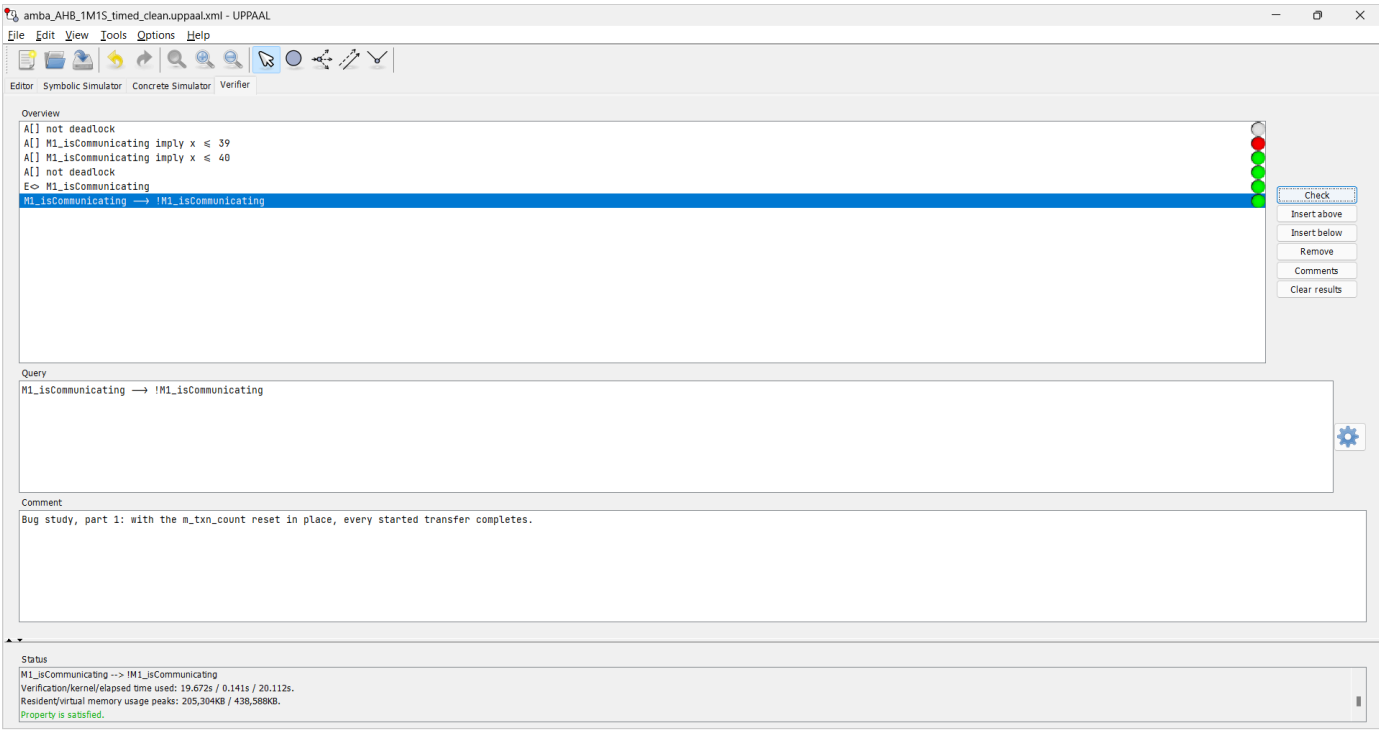



**Fig. 9.** All checks on the timed model. Adding the clock does not introduce a deadlock, communication does start, and with the `m_txn_count` reset in place every started transfer completes (`M1_isCommunicating --> !M1_isCommunicating`, **satisfied**).

## D. Bug study: the missing `m_txn_count` reset

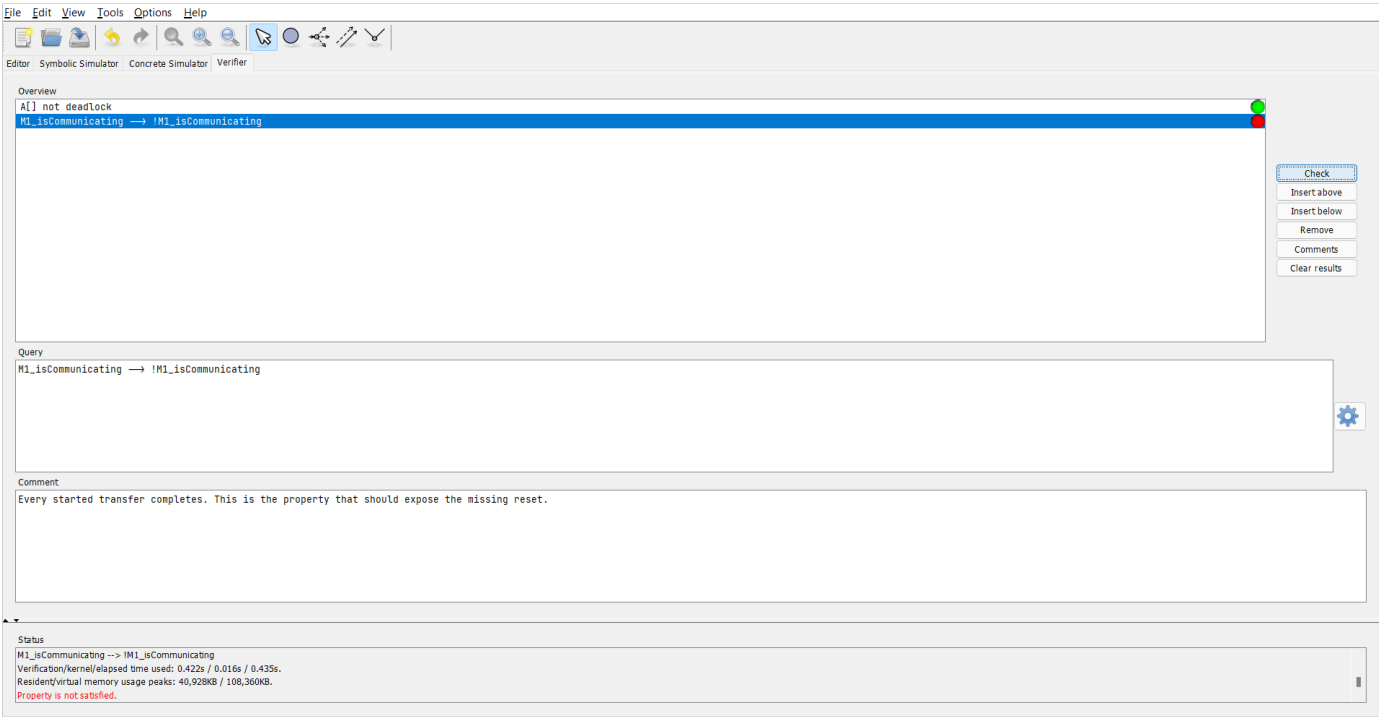



**Fig. 10.** With the reset removed: `A[] not deadlock` is still **satisfied**, but `M1_isCommunicating --> !M1_isCommunicating` is **not satisfied**. The bug shows up as a transfer that never finishes, not as a deadlock.

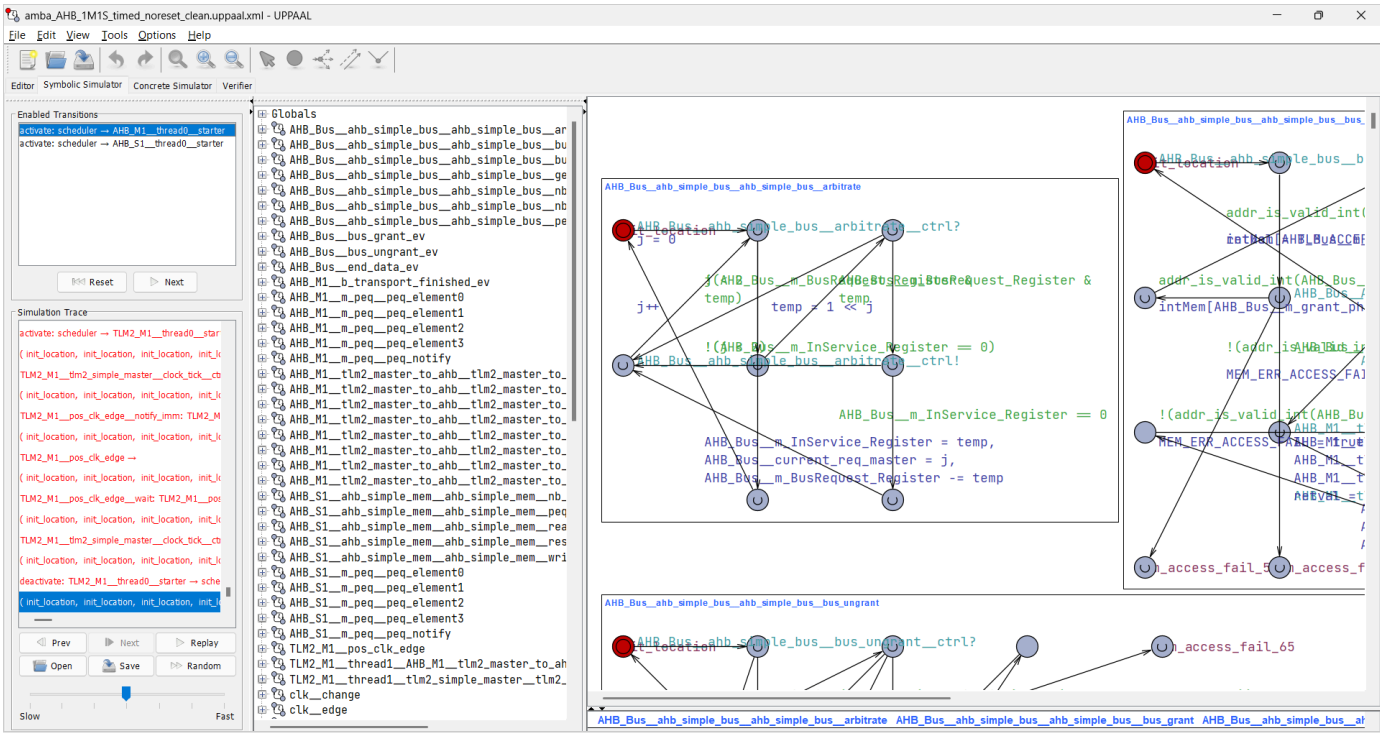



**Fig. 11.** The counterexample loaded into the Symbolic Simulator: the simulation trace (left), the variable values (middle) and the current location of each automaton in red (right).

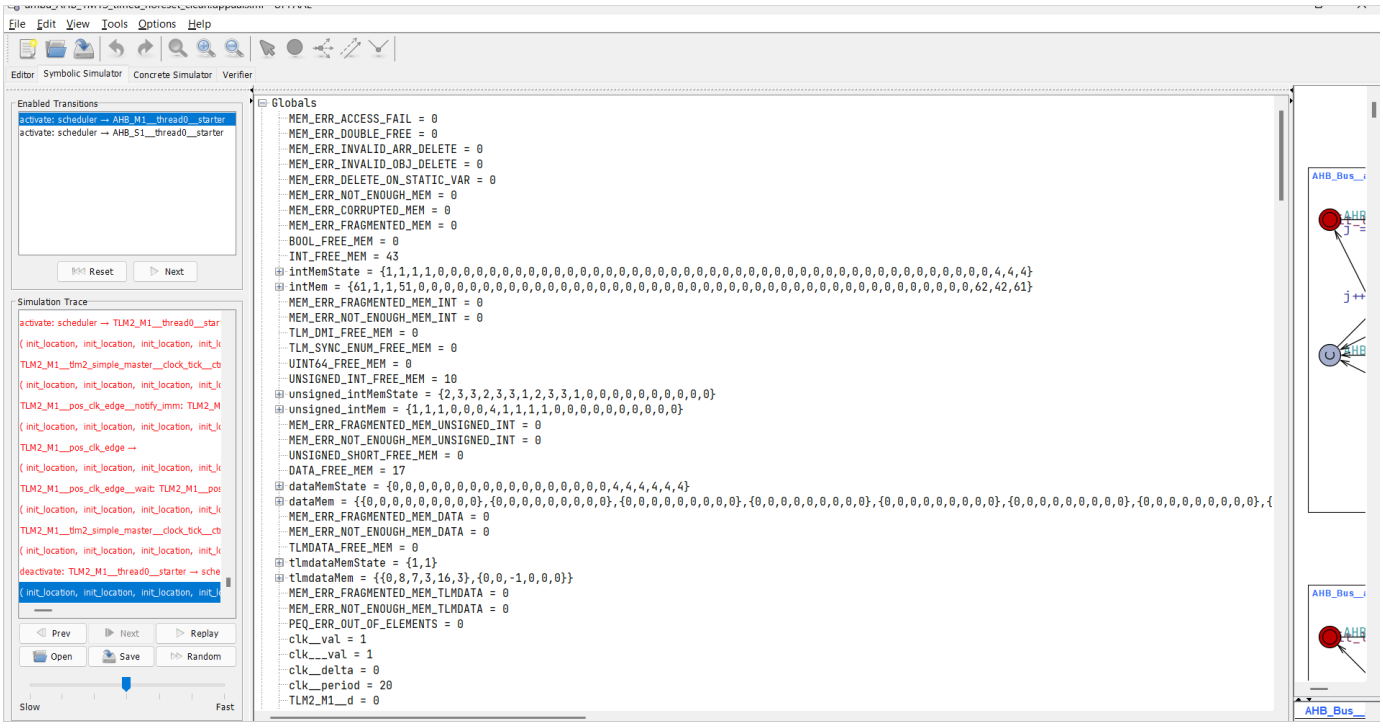
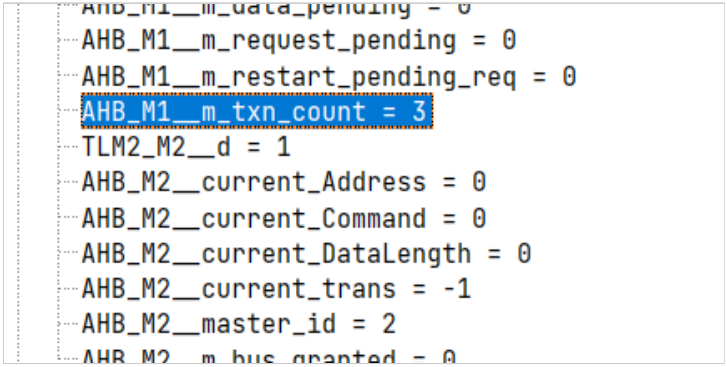
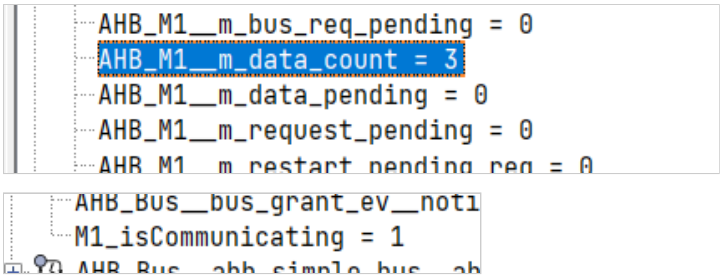
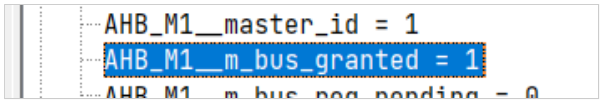



**Fig. 12.** Global variables at the end of the counterexample. All memory-error flags are 0, and `clk__period = 20`.









**Figs. 13–16.** Key values in the counterexample: `AHB_M1__m_data_count = 3` (all three beats transferred), but `AHB_M1__m_txn_count = 3` is never reset to 0, the master still holds the bus (`m_bus_granted = 1`) and `M1_isCommunicating = 1` forever. This is the missing-reset bug described in Section VI-B of the paper.